Name : Sarfraaz Shaikh

Enrollment Number : A71004824054

Subject : Machine Learning Lab

INDEX

LAB EXPERIMENT

Sr. No.	Title / Experiment Name
1	Plotting Graphs Using *Matplotlib* (Histogram, Bar Graph, Pie Chart, Box & Whisker Plot, Scatter Plot)
2	Computing Statistical Measures (Mean, Mode, Median, Range, Variance, Standard Deviation) Using *NumPy & Pandas*
3	Building Confusion Matrix and Calculating Accuracy, Precision, Recall, Specificity & F1-Score
4	Building a Decision Tree Using **ID3 Algorithm** (Using Standard Dataset)
5	Building a Decision Tree Using **CART Algorithm** (Using Standard Dataset)
6	Building a **Random Forest Classifier** (Using Standard Dataset)
7	Implementing **Simple Linear Regression** in Python (Using Standard Dataset)
8	Implementing **Naïve Bayes Classifier** in Python (Using Standard Dataset)
9	Implementing **Support Vector Machine (SVM)** Classifier in Python
10	Implementing **K-Means Clustering** in Python (Using Standard Dataset)

1. Considering appropriate datasets from standard dataset. Write a program to plot the following graphs using matplotlib libraries in python. (i) Histogram (ii) Bar Graphs (iii) Pie Charts (iv) Box and Whisker Plot (v) Scatter Plot

In [1]:
```
import pandas as pd

# Sample data
data = {
    "Sr. No.": list(range(1, 11)),
    "Student Name": ["Alice", "Bob", "Charlie", "Diana", "Ethan", "Fiona", "George", "Hannah", "Ian", "Julia"],
    "Sub1": [78, 65, 95, 55, 80, 90, 60, 70, 85, 92],
    "Sub2": [85, 70, 92, 60, 78, 85, 58, 75, 80, 94],
    "Sub3": [90, 72, 88, 58, 85, 89, 55, 72, 79, 95],
    "Sub4": [82, 68, 91, 65, 88, 92, 63, 78, 82, 91],
    "Sub5": [88, 60, 90, 62, 84, 94, 61, 76, 84, 93],
}

df = pd.DataFrame(data)
df["Total"] = df[["Sub1", "Sub2", "Sub3", "Sub4", "Sub5"]].sum(axis=1)
df["Percentage"] = df["Total"] / 5
df["Grade"] = df["Percentage"].apply(lambda x: "A+" if x >= 90 else ("A" if x >= 80 else ("B+" if x >= 70 else

# Save to CSV
df.to_csv("students_data.csv", index=False)
```

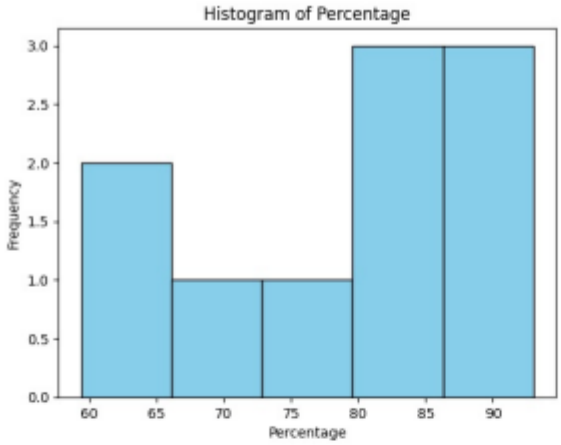
In [2]:
```
import pandas as pd

df = pd.read_csv("students_data.csv")
print(df)
```

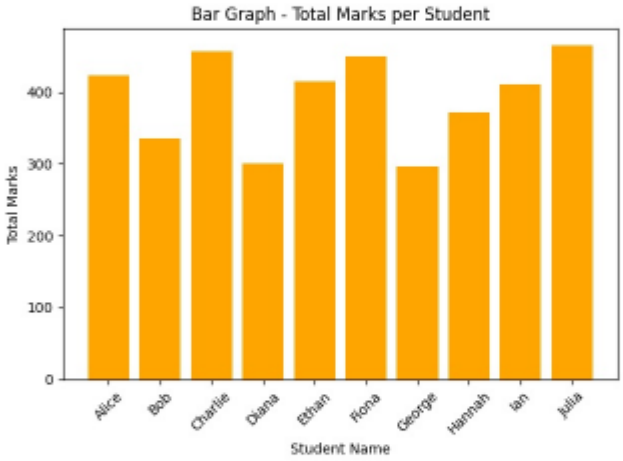
```
   Sr. No. Student Name  Sub1  Sub2  Sub3  Sub4  Sub5  Total  Percentage Grade
0        1        Alice    78    85    90    82    88    423        84.6     A
1        2          Bob    65    70    72    68    60    335        67.0     B
2        3      Charlie    95    92    88    91    90    456        91.2    A+
3        4        Diana    55    60    58    65    62    300        60.0     B
4        5        Ethan    80    78    85    88    84    415        83.0     A
5        6        Fiona    90    85    89    92    94    450        90.0    A+
6        7       George    60    58    55    63    61    297        59.4     C
7        8       Hannah    70    75    72    78    76    371        74.2    B+
8        9          Ian    85    80    79    82    84    410        82.0     A
9       10        Julia    92    94    95    91    93    465        93.0    A+
```

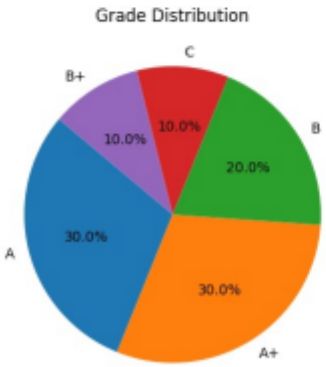
In [3]:
```
import matplotlib.pyplot as plt

# Histogram of percentages
plt.hist(df["Percentage"], bins=5, color='skyblue', edgecolor='black')
plt.title("Histogram of Percentage")
plt.xlabel("Percentage")
plt.ylabel("Frequency")
plt.show()

# Bar Graph of total marks
plt.bar(df["Student Name"], df["Total"], color='orange')
plt.title("Bar Graph - Total Marks per Student")
```

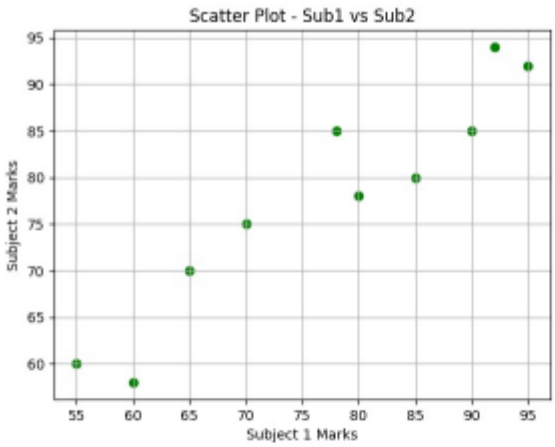

```
plt.hist(df["Percentage"], bins=5, color='skyblue', edgecolor='black')
plt.title("Histogram of Percentage")
plt.xlabel("Percentage")
plt.ylabel("Frequency")
plt.show()

# Bar Graph of total marks
plt.bar(df["Student Name"], df["Total"], color='orange')
plt.title("Bar Graph - Total Marks per Student")
```


```
plt.xlabel("Student Name")
plt.ylabel("Total Marks")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Pie chart of grades
grade_counts = df["Grade"].value_counts()
plt.pie(grade_counts, labels=grade_counts.index, autopct='%1.1f%%', startangle=140)
plt.title("Grade Distribution")
plt.show()

# Scatter plot - Sub1 vs Sub2
plt.scatter(df["Sub1"], df["Sub2"], color='green')
plt.title("Scatter Plot - Sub1 vs Sub2")
plt.xlabel("Subject 1 Marks")
plt.ylabel("Subject 2 Marks")
plt.grid(True)
plt.show()
```


Histogram of Percentage
Frequency
Percentage
0.0
0.5
1.0
1.5
2.0
2.5
3.0
60
65
70
75
80
85
90

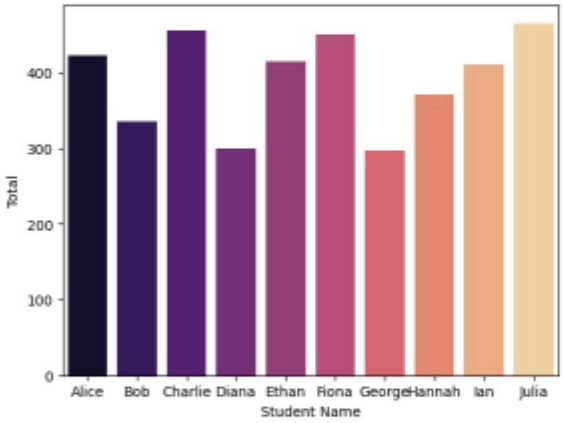

Bar Graph - Total Marks per Student
Total Marks
Student Name
0
100
200
300
400
Alice
Bob
Charlie
Diana
Ethan
Fiona
George
Hannah
Ian
Julia


Grade Distribution
C
B+
B
A
A+
10.0%
10.0%
20.0%
30.0%
30.0%



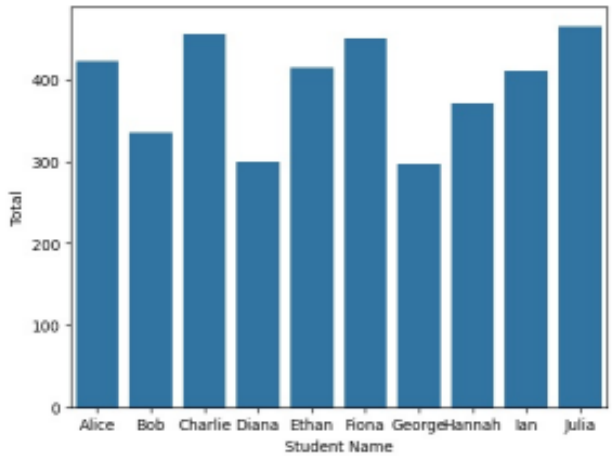

```
In [12]: sns.barplot(x="Student Name", y="Total", data=df, hue="Student Name", palette="magma", legend=False)
```

```
Out[12]: <Axes: xlabel='Student Name', ylabel='Total'>
```




```
In [13]: sns.barplot(x="Student Name", y="Total", data=df)
```

```
Out[13]: <Axes: xlabel='Student Name', ylabel='Total'>
```




```
In [ ]:
```

2. Considering appropriate datasets from standard dataset. Write a python program to compute the following operations using Numpy and Pandas library. 1) Mean 2) Mode 3) Median 4) Range 5) Variance 6) Standard deviation

csvReadinPython

September 20, 2025

[2]:
```
import pandas as pd
```

[9]:
```
df = pd.read_csv("C:\\Users\\Sarfraaz shaikh\\Documents\\Book1.csv")

df
```

[9]:

	Sr. No.	Student Name	Sub1	Sub2	Sub3	Sub4	Sub5	Total	Percentage	Grade
0	1	Ayaan	78	85	69	74	80	386	77.2	B
1	2	Meera	92	88	95	90	93	458	91.6	A
2	3	Rohan	65	70	68	72	66	341	68.2	C
3	4	Kavya	81	79	84	77	80	401	80.2	B
4	5	Arjun	55	60	58	62	57	292	58.4	D
5	6	Neha	88	91	86	90	89	444	88.8	A
6	7	Ishaan	72	75	70	74	73	364	72.8	C
7	8	Simran	95	92	94	96	93	470	94.0	A
8	9	Aman	60	65	63	62	61	311	62.2	D
9	10	Priya	85	88	84	86	87	430	86.0	B

[11]:
```
import numpy as np
```

[12]:
```
from scipy import stats
```

[13]:
```
df = pd.read_csv("C:\\Users\\Sarfraaz shaikh\\Documents\\Book1.csv")
```

[14]:
```
print("Dataset:")
print(df)
```

```
Dataset:
   Sr. No. Student Name  Sub1  Sub2  Sub3  Sub4  Sub5  Total  Percentage Grade
0        1        Ayaan    78    85    69    74    80    386        77.2     B
1        2        Meera    92    88    95    90    93    458        91.6     A
2        3        Rohan    65    70    68    72    66    341        68.2     C
3        4        Kavya    81    79    84    77    80    401        80.2     B
4        5        Arjun    55    60    58    62    57    292        58.4     D
5        6         Neha    88    91    86    90    89    444        88.8     A
6        7      Ishaan    72    75    70    74    73    364        72.8     C
7        8      Simran    95    92    94    96    93    470        94.0     A
8        9         Aman    60    65    63    62    61    311        62.2     D
9       10        Priya    85    88    84    86    87    430        86.0     B
```

```
[15]: # b) Select only subject marks for calculations
      marks = df[["Sub1", "Sub2", "Sub3", "Sub4", "Sub5"]].values.flatten()

[16]: mean_value = np.mean(marks)

[17]: mode_value = stats.mode(marks, keepdims=True).mode[0]

[18]: median_value = np.median(marks)

      # IV. Range
      range_value = np.ptp(marks)    # max - min

      # V. Variance
      variance_value = np.var(marks)

      # VI. Standard Deviation
      std_dev_value = np.std(marks)

[19]: print("\nStatistical Results:")
      print(f"Mean: {mean_value:.2f}")
      print(f"Mode: {mode_value}")
      print(f"Median: {median_value}")
      print(f"Range: {range_value}")
      print(f"Variance: {variance_value:.2f}")
      print(f"Standard Deviation: {std_dev_value:.2f}")

      Statistical Results:
      Mean: 77.94
      Mode: 88
      Median: 79.5
      Range: 41
      Variance: 144.98
      Standard Deviation: 12.04

[ ]:
```

3. Considering appropriate datasets from standard data. Write a python program to built confusion matrix and evaluate the followings. 1) Accuracy 2) Precision 3) Recall 4) Specificity 5) F1-Score

```python
# sarfraaz shaikh
# Enrollment number 054
import numpy as np
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
actual = ['Dog','Dog','Dog','Not Dog','Dog','Not Dog','Dog','Dog','Not Dog','Not Dog']
predicted = ['Dog','Not Dog','Dog','Not Dog','Dog','Dog','Dog','Dog','Not Dog','Not Dog']
labels = ['Dog', 'Not Dog']
cm = confusion_matrix(actual, predicted, labels=labels)
print("Confusion Matrix:\n", cm)
accuracy = accuracy_score(actual, predicted)
precision = precision_score(actual, predicted, pos_label="Dog")
recall = recall_score(actual, predicted, pos_label="Dog")
f1 = f1_score(actual, predicted, pos_label="Dog")
tn, fp, fn, tp = cm.ravel()
specificity = tn / (tn + fp)
print("\nMetrics:")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-Score: {f1:.2f}")
print(f"Specificity: {specificity:.2f}")
```

```
Confusion Matrix:
 [[5 1]
 [1 3]]

Metrics:
Accuracy: 0.80
Precision: 0.83
Recall: 0.83
F1-Score: 0.83
Specificity: 0.83
```

4. Using an appropriate data set from standard dataset, write a python program to build the decision tree using ID3 algorithm.

1 Name : Sarfraaz Shaikh

2 Enrollment : A71004824054

[1]:
```
import pandas as pd

data = pd.read_csv("C:\\Users\\Sarfraaz shaikh\\Downloads\\ID3 Alogrithm Data␣
↪set.csv")
print(data)
```

```
  CustomerID AgeGroup IncomeLevel MaritalStatus PriorPurchase ProductInterest  \
0       C001    Young         Low        Single            No     Electronics
1       C002   Middle        High       Married           Yes         Fashion
2       C003   Senior      Medium       Married            No       Groceries
3       C004    Young      Medium        Single            No         Fashion
4       C005   Middle      Medium       Married           Yes     Electronics
5       C006   Senior        High       Married           Yes         Fashion
6       C007    Young         Low        Single            No       Groceries
7       C008   Middle      Medium        Single           Yes     Electronics
8       C009   Senior      Medium       Married            No     Electronics
9       C010    Young        High       Married           Yes         Fashion

  Purchase
0       No
1      Yes
2      Yes
3       No
4      Yes
5      Yes
6       No
7      Yes
8       No
9      Yes
```

[16]:
```
def entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    entropy_val = -np.sum([
        (counts[i]/np.sum(counts)) * np.log2(counts[i]/np.sum(counts))
```

```python
        for i in range(len(elements))
    ])
    return entropy_val

# Step 3 : Calculate Information Gain
def information_gain(df, split_attribute, target_name="Purchase"):
    total_entropy = entropy(df[target_name])
    values, counts = np.unique(df[split_attribute], return_counts=True)
    weighted_entropy = np.sum([
        (counts[i]/np.sum(counts)) * entropy(df[df[split_attribute] ==␣
 ↪values[i]][target_name])
        for i in range(len(values))
    ])
    info_gain = total_entropy - weighted_entropy
    return info_gain

# Step 4 : Define Tree Node and ID3 Algorithm
class TreeNode:
    def __init__(self, attribute=None, is_leaf=False, classification=None):
        self.attribute = attribute
        self.is_leaf = is_leaf
        self.classification = classification
        self.children = {}

def id3(df, target_name="Purchase", attributes=None):
    if attributes is None:
        attributes = df.columns.drop(target_name)

    # Case 1: All target values are the same
    if len(np.unique(df[target_name])) == 1:
        return TreeNode(is_leaf=True, classification=np.
 ↪unique(df[target_name])[0])

    # Case 2: No attributes left
    if len(attributes) == 0:
        return TreeNode(is_leaf=True, classification=df[target_name].mode()[0])

    # Calculate info gain for all attributes
    gains = {attr: information_gain(df, attr, target_name) for attr in␣
 ↪attributes}
    best_attr = max(gains, key=gains.get)

    # Create root node for this attribute
    root = TreeNode(attribute=best_attr)
```

```python
        subset = df[df[best_attr] == value]
        if subset.empty:
            root.children[value] = TreeNode(is_leaf=True,␣
↪classification=df[target_name].mode()[0])
        else:
            remaining_attrs = [attr for attr in attributes if attr != best_attr]
            root.children[value] = id3(subset, target_name, remaining_attrs)

    return root

# Step 5 : Print the Decision Tree
def print_tree(node, level=0):
    indent = "  " * level
    if node.is_leaf:
        print(f"{indent}Predict → {node.classification}")
    else:
        print(f"{indent}[{node.attribute}]")
        for value, child in node.children.items():
            print(f"{indent}-- {value} -->")
            print_tree(child, level + 1)

# Step 6 : Build and Print Tree
print(" DECISION TREE STRUCTURE:\n")
tree = id3(df)
print_tree(tree)
```

```
 DECISION TREE STRUCTURE:

[PriorPurchase]
-- No -->
  [AgeGroup]
  -- Senior -->
    [ProductInterest]
    -- Electronics -->
      Predict → No
    -- Groceries -->
      Predict → Yes
  -- Young -->
    Predict → No
-- Yes -->
  Predict → Yes
```

5.Using an appropriate data set from standard dataset, write a python program to build the decision tree using CART algorithm.

October 11, 2025

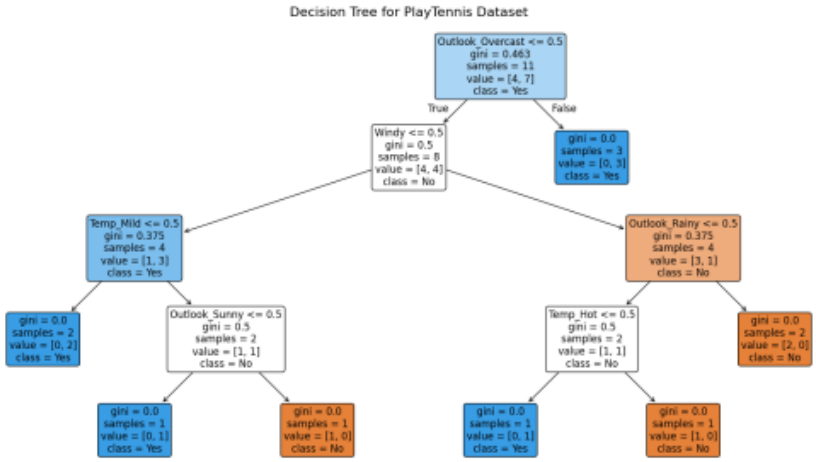
[ ]:
```
#  Name :Sarfraaz Shaikh
```

[10]:
```
# Enrollment number 054
```

[ ]:
```
# Cart Algorithm
```

[9]:
```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("C:\\Users\\Sarfraaz shaikh\\Downloads\\Tennis.csv")

# Strip any whitespace from column names
df.columns = df.columns.str.strip()

# Encode the target variable (assumed to be the last column)
target_column = df.columns[-1]
le = LabelEncoder()
df[target_column] = le.fit_transform(df[target_column])  # e.g., Yes/No -> 1/0

# Separate features and target
X = df.drop(target_column, axis=1)
y = df[target_column]

# One-hot encode categorical input features
X = pd.get_dummies(X)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

# Train Decision Tree Classifier (CART)
clf = DecisionTreeClassifier(criterion='gini', random_state=0)
clf.fit(X_train, y_train)
```

```python
# Plot the decision tree
plt.figure(figsize=(20, 10))
plot_tree(clf,
          feature_names=X.columns,
          class_names=le.classes_,
          filled=True,
          rounded=True,
          fontsize=12)
plt.title("Decision Tree for PlayTennis Dataset", fontsize=16)
plt.show()
```




```python
[ ]:
```

6. Using an appropriate data set from standard dataset, write a python program to build the decision tree using Random Forest algorithm.

1 Name : Sarfraaz Shaikh

2 Enrollment Number : A71004824054

[1]:
```
import pandas as pd

data = pd.read_csv("C:\\Users\\Sarfraaz shaikh\\Downloads\\Random Forest.csv")
print(data)
```

```
   ApplicantID  Age  Salary  LoanAmount LoanApproved
0         A001   25      30          10            1
1         A002   32      45          20            0
2         A003   40      60          25            1
3         A004   29      75          40            1
4         A005   35      38          15            0
5         A006   42      55          35            1
6         A007   27      50          22            0
7         A008   39      85          45            1
8         A009   31      65          30            1
9         A010   28      48          18            0
10        A011   33      70          20            ?
```

[2]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

[3]:
```
data = {
    'Age': [25, 32, 40, 29, 35, 42, 27, 39, 31, 28],
    'Salary': [30, 45, 60, 75, 38, 55, 50, 85, 65, 48],
    'LoanAmount': [10, 20, 25, 40, 15, 35, 22, 45, 30, 18],
    'LoanApproved': [1, 0, 1, 1, 0, 1, 0, 1, 1, 0]
}
```

[4]:
```
df = pd.DataFrame(data)
print(" CREDIT RISK DATASET:")
print(df)
```

```
CREDIT RISK DATASET:
   Age  Salary  LoanAmount  LoanApproved
0   25      30          10             1
1   32      45          20             0
2   40      60          25             1
3   29      75          40             1
4   35      38          15             0
5   42      55          35             1
6   27      50          22             0
7   39      85          45             1
8   31      65          30             1
9   28      48          18             0
```

[5]:
```
X = df[['Age', 'Salary', 'LoanAmount']]
y = df['LoanApproved']
```

[6]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

[7]:
```
model = RandomForestClassifier(n_estimators=10, random_state=42)
model.fit(X_train, y_train)
```

[7]:
```
RandomForestClassifier(n_estimators=10, random_state=42)
```

[8]:
```
y_pred = model.predict(X_test)
```

[9]:
```
accuracy = accuracy_score(y_test, y_pred)
print("\n Model Evaluation:")
print(f"Accuracy: {accuracy:.2f}")
```

```
 Model Evaluation:
Accuracy: 1.00
```

[ ]:
```
new_applicant = [[33, 70, 20]]
loan_approval = model.predict(new_applicant)
loan_probability = model.predict_proba(new_applicant)[:, 1][0]
```

[11]:
```
print("\n Prediction for new applicant:")
print(f"Loan Approved: {loan_approval[0]} (1 = Approved, 0 = Not Approved)")
print(f"Probability of Approval: {loan_probability:.2f}")
```

```
 Prediction for new applicant:
Loan Approved: 0 (1 = Approved, 0 = Not Approved)
Probability of Approval: 0.40
```

7. Write a program in python to implement a simple linear regression model using standard dataset.

October 29, 2025

1 Sarfraaz Shaikh

2 Enrollment Number is A71004824054

[2]:
```
import pandas as pd
```

[3]:
```
import matplotlib.pyplot as plt
```

[5]:
```
from sklearn.linear_model import LinearRegression
```

[3]:
```
import pandas as pd
data={
    "Temperature":[5,7,10,12,15,17,20,22,25,27],
    "Jackets Sold":[90,85,80,70,65,60,55,48,42,38]
}
df =pd.DataFrame(data)
print(df)
```

```
   Temperature  Jackets Sold
0            5            90
1            7            85
2           10            80
3           12            70
4           15            65
5           17            60
6           20            55
7           22            48
8           25            42
9           27            38
```

[4]:
```
X =df[["Temperature"]]  # Independent variable
```

[5]:
```
Y = df["Jackets Sold"] # Dependent variable
```

[7]:
```
from sklearn.linear_model import LinearRegression
model =LinearRegression()
```

[12]:
```
model.fit(X,Y)
b1 = model.coef_[0]
```

[14]:
```
b0=model.intercept_
```

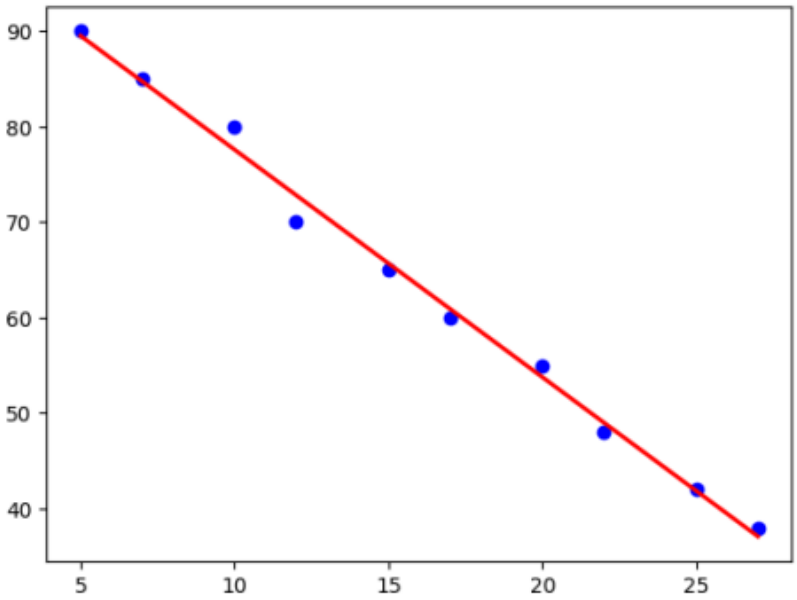
[15]:
```
print(f"Equation of line: Y = {b0:.2f} + ({b1:.2f})X")
```

```
Equation of line: Y = 101.45 + (-2.38)X
```

[22]:
```
Y_pred= model.predict(X)
```

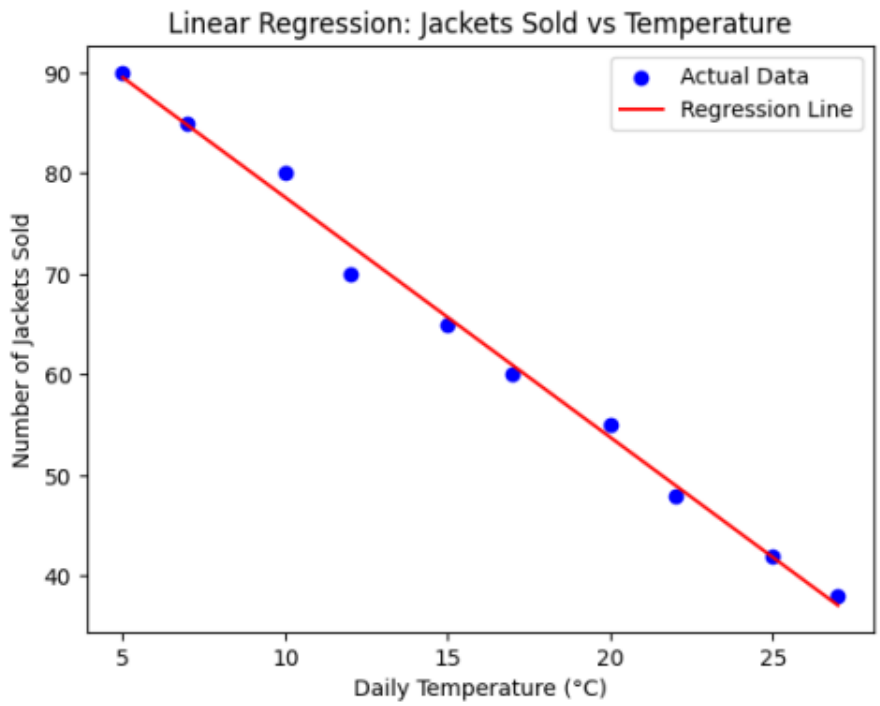
[23]:
```
import matplotlib.pyplot as plt


plt.scatter(X, Y, color='blue', label='Actual Data')
plt.plot(X, Y_pred, color='red', linewidth=2, label='Regression Line')
```

[23]:
```
[<matplotlib.lines.Line2D at 0x13383c96ad0>]
```



[36]:
```
import numpy as np
from sklearn.linear_model import LinearRegression

# Scatter plot of original data
```

```python
plt.scatter(df["Temperature"], df["Jackets Sold"], color='blue', label="Actual␣
↪Data")

# Plot regression line
plt.plot(df["Temperature"], model.predict(df[["Temperature"]]), color='red',␣
↪label="Regression Line")

# Add title and axis labels
plt.title("Linear Regression: Jackets Sold vs Temperature")
plt.xlabel("Daily Temperature (°C)")
plt.ylabel("Number of Jackets Sold")

# Show legend
plt.legend()

# Display plot
plt.show()

# Predict for a new temperature value
new_temp = np.array([[30]])
predicted_sales = model.predict(new_temp)
print(f"Predicted Jackets Sold at {new_temp[0][0]}°C: {predicted_sales[0]:.2f}")
```



8. Write a program in python to implement Naïve Bayes classifier model using standard dataset.

November 21, 2025

1 Name : Sarfraaz Shaikh

2 Enrollment Number : A71004824054

[1]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder
```

[2]:
```
data = {
    'Outlook': ['Rainy', 'Sunny', 'Overcast', 'Overcast', 'Sunny', 'Rainy',␣
 ↪'Sunny', 'Overcast', 'Rainy', 'Sunny', 'Sunny', 'Rainy', 'Overcast',␣
 ↪'Overcast'],
    'Play': ['Yes', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'No', 'No',␣
 ↪'Yes', 'No', 'Yes', 'Yes']
}
```

[3]:
```
df = pd.DataFrame(data)
```

[4]:
```
P_Yes = len(df[df['Play'] == 'Yes']) / len(df)
P_No = len(df[df['Play'] == 'No']) / len(df)
```

[5]:
```
Sunny_Yes_count = len(df[(df['Outlook'] == 'Sunny') & (df['Play'] == 'Yes')])
Sunny_No_count = len(df[(df['Outlook'] == 'Sunny') & (df['Play'] == 'No')])
```

[6]:
```
P_Sunny_given_Yes = Sunny_Yes_count / len(df[df['Play'] == 'Yes'])
P_Sunny_given_No = Sunny_No_count / len(df[df['Play'] == 'No'])
```

[7]:
```
Posterior_Yes = P_Yes * P_Sunny_given_Yes
Posterior_No = P_No * P_Sunny_given_No
```

[8]:
```
# Output the results
print("### 1. Naive Bayes Prediction (Manual Calculation) ###")
print(f"P(Play=Yes) * P(Sunny|Play=Yes) = ({P_Yes:.3f}) * ({P_Sunny_given_Yes:.
 ↪3f}) = {Posterior_Yes:.4f}")
print(f"P(Play=No) * P(Sunny|Play=No) = ({P_No:.3f}) * ({P_Sunny_given_No:.3f})␣
 ↪= {Posterior_No:.4f}")
```

```
### 1. Naive Bayes Prediction (Manual Calculation) ###
P(Play=Yes) * P(Sunny|Play=Yes) = (0.714) * (0.300) = 0.2143
P(Play=No) * P(Sunny|Play=No) = (0.286) * (0.500) = 0.1429
```

[9]:
```
if Posterior_Yes > Posterior_No:
    Prediction = 'Yes'
else:
    Prediction = 'No'

print(f"\nPrediction for Outlook=Sunny is: **{Prediction}** (since␣
 ↪{Posterior_Yes:.4f} > {Posterior_No:.4f})\n")
```

```
Prediction for Outlook=Sunny is: **Yes** (since 0.2143 > 0.1429)
```

[10]:
```
print("---")
print("### 2. Naive Bayes Prediction (Scikit-learn) ###")
```

```
---
### 2. Naive Bayes Prediction (Scikit-learn) ###
```

[11]:
```
X = df[['Outlook']]
y = df['Play']
```

[12]:
```
encoder = OrdinalEncoder()
X_encoded = encoder.fit_transform(X)
y_encoded = pd.Series(y).apply(lambda x: 1 if x == 'Yes' else 0).values
```

[13]:
```
model = CategoricalNB()
model.fit(X_encoded, y_encoded)
```

[13]:
```
CategoricalNB()
```

[14]:
```
categories = encoder.categories_[0]
sunny_index = list(categories).index('Sunny')
```

[15]:
```
X_test = [[sunny_index]]

prediction_encoded = model.predict(X_test)[0]
```

[16]:
```
prediction_label = 'Yes' if prediction_encoded == 1 else 'No'
```

[17]:
```
probabilities = model.predict_proba(X_test)[0]

print(f"Encoded Categories: {categories}")
print(f"Encoded 'Sunny' input: {X_test}")
print(f"Prediction (Encoded): {prediction_encoded}")
```

```
print(f"Predicted probabilities: P(No)={probabilities[0]:.4f},␣
↪P(Yes)={probabilities[1]:.4f}")
print(f"Scikit-learn Prediction for Outlook=Sunny is: **{prediction_label}**")
```

```
Encoded Categories: ['Overcast' 'Rainy' 'Sunny']
Encoded 'Sunny' input: [[2]]
Prediction (Encoded): 1
Predicted probabilities: P(No)=0.3578, P(Yes)=0.6422
Scikit-learn Prediction for Outlook=Sunny is: **Yes**
```

9.Write a program in python to implement SVM classifier model using standard dataset.

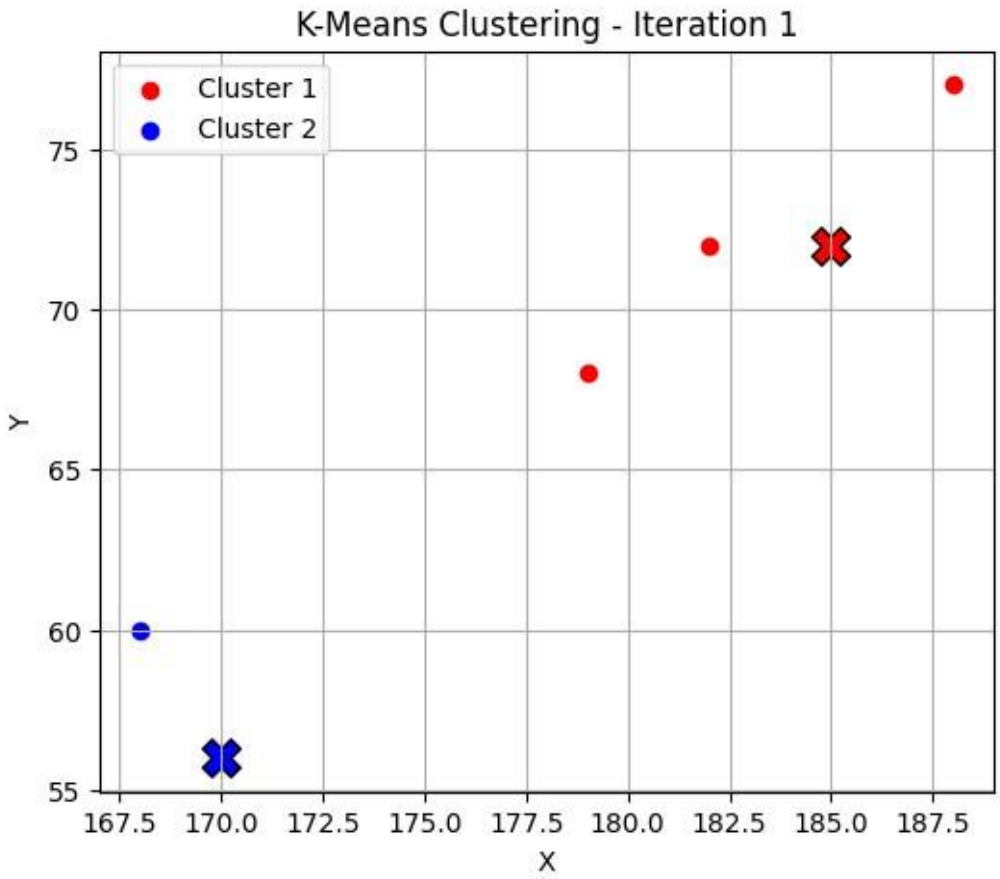
10. Write a program in python to implement K-Means Clustering using standard dataset

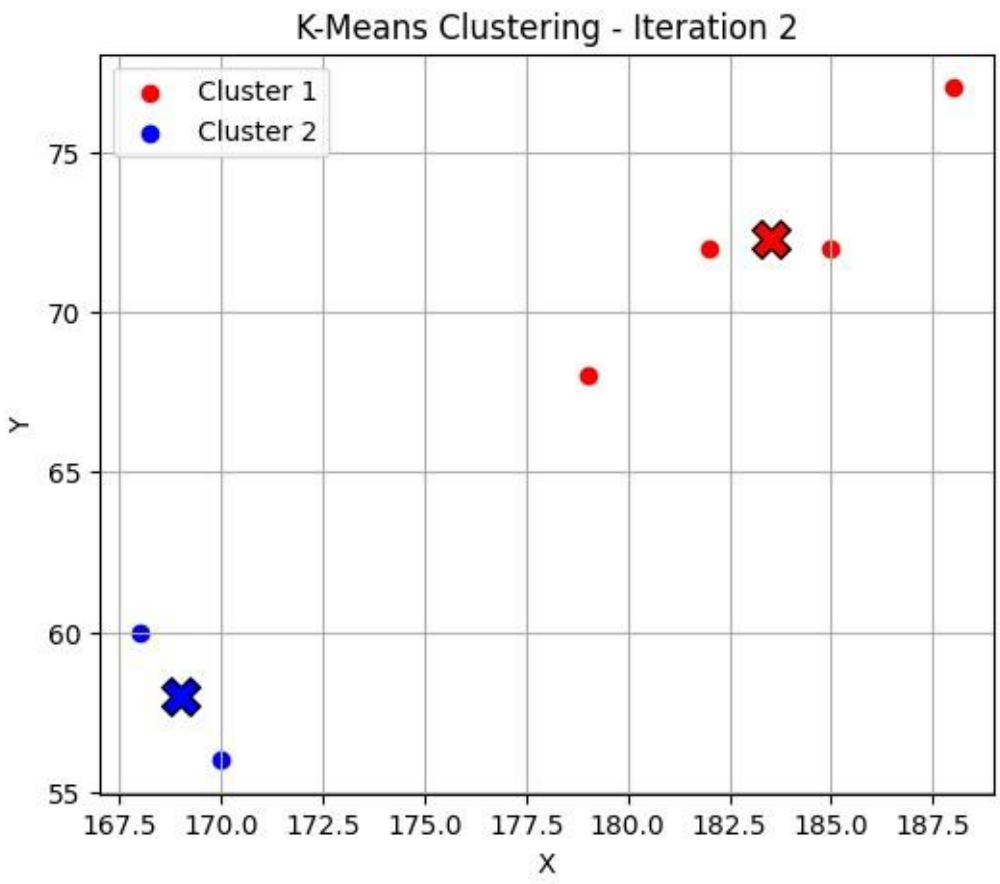
→

[1]:
```
# K Means Algorithm with plot
import numpy as np
import matplotlib.pyplot as plt
# Data points
data = np.array([
    [185, 72],
    [170, 56],
    [168, 60],
    [179, 68],
    [182, 72],
    [188, 77]
])
# Initial centroids: first two data points
centroids = np.array([data[0], data[1]])
# Assign each point to the nearest centroid
def assign_clusters(data, centroids):
    distances = np.linalg.norm(data[:, np.newaxis] - centroids, axis=2)
    return np.argmin(distances, axis=1)
# Update centroids by calculating the mean of each cluster
def update_centroids(data, labels, k):
    return np.array([data[labels == i].mean(axis=0) for i in range(k)])
```

[2]:
```
# Plotting function
def plot_clusters(data, labels, centroids, iteration):
    colors = ['red', 'blue']
    plt.figure(figsize=(6, 5))
    for i in range(2):
        cluster_points = data[labels == i]
        plt.scatter(cluster_points[:, 0], cluster_points[:, 1], c=colors[i],␣
↪label=f'Cluster {i+1}')
        plt.scatter(centroids[i][0], centroids[i][1], c=colors[i], marker='X',␣
↪s=200, edgecolor='black')
    plt.title(f'K-Means Clustering - Iteration {iteration}')
```

```
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.legend()
    plt.grid(True)
    plt.show()

# Run K-Means for 2 iterations and plot
for i in range(1, 3):
    labels = assign_clusters(data, centroids)
    plot_clusters(data, labels, centroids, i)
    centroids = update_centroids(data, labels, k=2)
```




K-Means Clustering - Iteration 2
Cluster 1
Cluster 2
75
70
65
60
55
Y
167.5
170.0
172.5
175.0
177.5
180.0
182.5
185.0
187.5
X